TAREA MI PRIMER GRAFO

ALEJANDRO LÓPEZ CLARO

Crea el grafo según la imagen adjunta y realiza las siguientes consultas:

- Mostrar todas las ciudades
- Contar cuántas ciudades hay
- Listar todas las carreteras con su distancia
- Contar cuántas carreteras hay en total

Reto 1:
Amazon quiere abrir un nuevo centro logístico y busca una ciudad con el resto de España.
Objetivo: Encontrar la ciudad con mayor número de conexiones directas (mayor "grado" del nodo).

Reto 2:
No basta con tener muchas carreteras; queremos una ubicación que esté cerca (en número de saltos) de la mayoría de las demás ciudades.
Objetivo: Calcular para cada ciudad el número promedio de saltos hasta el resto (concepto de cercanía o centralidad). Cuanto menor sea la media de saltos, más central es la ciudad (buena candidata para el almacén principal).

Reto 3:
Queremos que el nuevo centro sirva tanto al norte como al sur de la península, reduciendo los tiempos de envío medios.
Objetivo: Seleccionar una ciudad cuyo camino medio hacia las del norte y las del sur sea similar.

1º PASO: CREACIÓN DE RESTRICCIONES

```
/*
   Mi primer grafo: ciudades de España y carreteras que las unen.
   Transforma el mapa del enunciado en un grafo con 17 ciudades (nodos) y
23 carreteras (relaciones).
   El proceso se divide en tres fases: restricciones, nodos y relaciones.
*/

// Bloque 1: Restricciones e Índices

CREATE CONSTRAINT ciudad_nombre_unique IF NOT EXISTS FOR (c:Ciudad)
REQUIRE c.nombre IS UNIQUE;

```

2º PASO: CREACIÓN DE LOS NODOS (17 CIUDADES)

```
// Bloque 2: Creación de los nodos (17 ciudades)
// La propiedad "zona" solo se utiliza en el Reto 3 (norte / centro / sur)

UNWIND [
  {nombre: 'Coruña',     zona: 'norte'},
  {nombre: 'Vigo',       zona: 'norte'},
  {nombre: 'Oviedo',     zona: 'norte'},
  {nombre: 'Bilbao',     zona: 'norte'},
  {nombre: 'Valladolid', zona: 'norte'},
  {nombre: 'Zaragoza',   zona: 'norte'},
  {nombre: 'Barcelona',  zona: 'norte'},
  {nombre: 'Gerona',     zona: 'norte'},
  {nombre: 'Madrid',     zona: 'centro'},
  {nombre: 'Badajoz',    zona: 'sur'},
  {nombre: 'Albacete',   zona: 'sur'},
  {nombre: 'Valencia',   zona: 'sur'},
  {nombre: 'Murcia',     zona: 'sur'},
  {nombre: 'Jaén',       zona: 'sur'},
  {nombre: 'Granada',    zona: 'sur'},
  {nombre: 'Sevilla',    zona: 'sur'},
  {nombre: 'Cádiz',      zona: 'sur'}
] AS c
MERGE (:Ciudad {nombre: c.nombre, zona: c.zona});

```

3º PASO: CREACIÓN DE LAS RELACIONES (23 CARRETERAS)

```
// Bloque 3: Creación de las relaciones (23 carreteras)
// Cada fila es [ciudad origen, ciudad destino, kilómetros].
// Se crea una sola dirección; las consultas usan el patrón sin flecha para
recorrerlas en ambos sentidos.

UNWIND [
  ['Coruña',    'Vigo',       171],
  ['Coruña',    'Valladolid', 455],
  ['Vigo',      'Valladolid', 356],
  ['Oviedo',    'Bilbao',     304],
  ['Bilbao',    'Valladolid', 280],
  ['Bilbao',    'Madrid',     395],
  ['Bilbao',    'Zaragoza',   324],
  ['Valladolid','Madrid',     193],
  ['Zaragoza',  'Madrid',     325],
  ['Zaragoza',  'Barcelona',  296],
  ['Barcelona', 'Gerona',     100],
  ['Barcelona', 'Valencia',   349],
  ['Madrid',    'Badajoz',    403],
  ['Madrid',    'Albacete',   251],
  ['Madrid',    'Jaén',       335],
  ['Albacete',  'Valencia',   191],
  ['Albacete',  'Murcia',     150],
  ['Valencia',  'Murcia',     241],
  ['Jaén',      'Sevilla',    242],
  ['Jaén',      'Granada',     99],
  ['Sevilla',   'Granada',    256],
  ['Sevilla',   'Cádiz',      125],
  ['Granada',   'Murcia',     278]
] AS r
MATCH (a:Ciudad {nombre: r[0]}), (b:Ciudad {nombre: r[1]})
MERGE (a)-[:CARRETERA {km: r[2]}]->(b);

```

CONSULTA Nº 1: Mostrar todas las ciudades

```
// Consulta 1: Mostrar todas las ciudades

MATCH (c:Ciudad)
RETURN c.nombre AS ciudad
ORDER BY ciudad;
```

Table | Text | Code

	ciudad
12	"Oviedo"
13	"Sevilla"
14	"Valencia"
15	"Valladolid"
16	"Vigo"
17	"Zaragoza"

CONSULTA Nº2: Contar cuántas ciudades hay

```
// Consulta 2: Contar cuántas ciudades hay (esperado: 17)

MATCH (c:Ciudad)
RETURN count(c) AS total_ciudades;
```

Table | Text

	total_ciudades
1	17

CONSULTA Nº3: Listar todas las carreteras con su distancia

```
// Consulta 3: Listar todas las carreteras con su distancia

MATCH (a:Ciudad)-[r:CARRETERA]→(b:Ciudad)
RETURN a.nombre AS origen, b.nombre AS destino, r.km AS km
ORDER BY km DESC;
```

Table

	origen	destino	km
18	"Albacete"	"Valencia"	191
19	"Coruña"	"Vigo"	171
20	"Albacete"	"Murcia"	150
21	"Sevilla"	"Cádiz"	125
22	"Barcelona"	"Gerona"	100
23	"Jaén"	"Granada"	99

CONSULTA Nº4: Contar cuántas carreteras hay en total

```
// Consulta 4: Contar cuántas carreteras hay en total (esperado: 23)

MATCH ()-[r:CARRETERA]→()
RETURN count(r) AS total_carreteras;
```

Table

	total_carreteras
1	23

1º RETO:

```
// Reto 1: Ciudad con mayor número de conexiones directas (grado del nodo)
// El patrón sin flecha cuenta carreteras entrantes y salientes.

MATCH (c:Ciudad)-[r:CARRETERA]-()
RETURN c.nombre AS ciudad, count(r) AS conexiones
ORDER BY conexiones DESC, ciudad
LIMIT 1;
```

Table

	ciudad	conexiones
1	"Madrid"	6

2º RETO:

```cypher
// Reto 2: Cercanía. Número medio de saltos desde cada ciudad hasta el resto.
// shortestPath cuenta saltos (nº de carreteras), no kilómetros.
// Cuanto menor es la media, más central es la ciudad.

MATCH (a:Ciudad), (b:Ciudad)
WHERE a <> b
MATCH p = shortestPath((a)-[:CARRETERA*]-(b))
RETURN a.nombre AS ciudad,
       round(avg(length(p)) * 100) / 100 AS saltos_medios
ORDER BY saltos_medios ASC, ciudad;
```

Table | Text | Warn | Code

	ciudad	saltos_medios
1	"Madrid"	1.75
2	"Bilbao"	2.19
3	"Albacete"	2.25
4	"Jaén"	2.25
5	"Zaragoza"	2.25
6	"Valladolid"	2.31

3º RETO:

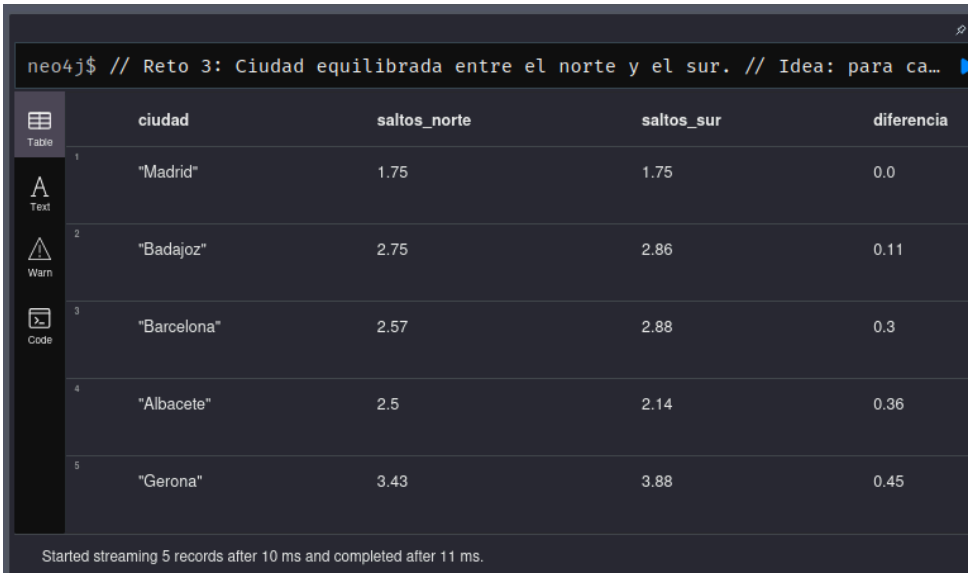
```cypher
// Reto 3: Ciudad equilibrada entre el norte y el sur.
// Idea: para cada ciudad, medir cuántos saltos hay de media hasta las ciudades
// del norte y hasta las del sur. La mejor es la que tiene las dos medias más parecidas.

// Paso 1: emparejar cada ciudad con todas las ciudades del norte o del sur
MATCH (origen:Ciudad), (destino:Ciudad)
WHERE origen <> destino
  AND destino.zona IN ['norte', 'sur']

// Paso 2: calcular el camino más corto (en saltos) entre cada pareja
MATCH camino = shortestPath((origen)-[:CARRETERA*]-(destino))

// Paso 3: media de saltos de cada ciudad hacia el norte y hacia el sur
WITH origen,
     avg(CASE WHEN destino.zona = 'norte' THEN length(camino) END) AS media_norte,
     avg(CASE WHEN destino.zona = 'sur'   THEN length(camino) END) AS media_sur

// Paso 4: mostrar el resultado, de más a menos equilibrada
RETURN origen.nombre AS ciudad,
       round(media_norte, 2) AS saltos_norte,
       round(media_sur, 2) AS saltos_sur,
       round(abs(media_norte - media_sur), 2) AS diferencia
ORDER BY diferencia
LIMIT 5;
```



VISUALIZACIÓN DEL GRAFO

